MANCHESTER 1824
The University of Manchester

MATH49111/69111: Scientific Computing

First lab session 2026/27

Matthias Heil

M.Heil@maths.manchester.ac.uk

Welcome to MATH49111/69111

- We'll teach you Scientific Computing and/in C++.
- The course is not "just" about writing code but also about the underlying mathematics (where applicable) that is being encoded. The relation between maths and code isn't 1:1.
- Key is to develop understanding!
  - algorithms and their robustness
  - debugging and critical assessment/interpretation of the results
  - coding style, error handling and documentation.
- No formal pre-requisites, so we'll teach everything from scratch, incl. the procedural part of the language. We assume that you've written some code in some (other) language before but not a real problem if not!

Format

- Weeks 1-5:
  - Recorded videos, accessible via the course's Canvas page. Keep watching at your own pace but make sure you keep up with (or stay ahead of!) the schedule.
  - Weekly two-hour computer lab classes where you can raise questions about the videos and/or the exercise sheets. These are run by myself, Mark Muldoon and PhD students.
- Week 6: Reading week.
- Weeks 7-12: Projects and weekly computer labs (they just continue at the normal time).

The main point of contact is the computer lab session, but you can discuss/raise issues on the Canvas Discussion tool.

Assessment

The course is assessed 100% by coursework.

- Exercise Sheets 1-4 have an "Analysis Exercise", each worth 2.5% of the overall grade. You're supposed to submit your solutions (on Tuesdays) via Gradescope; see below. The solution to Analysis Exercise N is due on Tuesday of Week $N+2$.
- We then have two bigger projects, one worth 30% the other 60%. Same format as the analysis exercises but "meatier".

Assessment

The course is assessed 100% by coursework.

- Exercise Sheets 1-4 have an "Analysis Exercise", each worth 2.5% of the overall grade. You're supposed to submit your solutions (on Tuesdays) via Gradescope; see below. The solution to Analysis Exercise N is due on Tuesday of Week $N+2$.
- We then have two bigger projects, one worth 30% the other 60%. Same format as the analysis exercises but "meatier".

All of the projects require the submissions of

- C++ code that does what it's supposed to do. Gradescope will check that it does! (0-2 marks)
- An accompanying powerpoint-style analysis of the results, following specific instructions given. There's a strict page limit! A human being will check that it contains what it's supposed to contain, and will award it 0-2 marks.
- The same human being will assess the code structure/layout/comments and will award 0-2 marks for this important aspect.

Details of the marking "rubrics" are in `Analysis_exercise_info.pdf`.

Assessment

- The folder *"Assessment: Rules and resources."* on the Canvas page provides much more information about what is required:
  - Reminders about plagiarism and collusion.
  - A few thoughts/edicts about ChatGPT.
  - Resources for writing reports, including
    - `Analysis_exercise_info.pdf`: Instructions for the Analysis Exercises and Projects.
    - Model solutions for a (dummy) "Analysis Exercise 0": We provide C++ code, the pdf file for the analysis, as well as the underlying latex source code, pdf files for the figures and the gnuplot script used to create them. You can use the C++ code and the report pdf file as test submissions to Gradescope.
    - `comments_in_code.pdf`: A discussion of how to comment/structure/annotate code.
    - A plotting guide, incl. a pdf file with specific instructions on how to create line (and other) plots in matlab.

Comments on coding, projects and reports

- Please read my comments on AI use! Talk/chat to anybody (human or non-human) but the work has to be your own and you have to understand it! Note that, in their infinite wisdom, the university enable Copilot by default, but don't blindly accept its suggestions.

Comments on coding, projects and reports

- Please read my comments on AI use! Talk/chat to anybody (human or non-human) but the work has to be your own and you have to understand it! Note that, in their infinite wisdom, the university enable Copilot by default, but don't blindly accept its suggestions.
- Gradescope is not a debugger! Passing our unit tests is not the same as your code being right/efficient/sensible/... We will read it and mark you down for inefficient/unclear/non-robust code.

Comments on coding, projects and reports

- Please read my comments on AI use! Talk/chat to anybody (human or non-human) but the work has to be your own and you have to understand it! Note that, in their infinite wisdom, the university enable Copilot by default, but don't blindly accept its suggestions.
- Gradescope is not a debugger! Passing our unit tests is not the same as your code being right/efficient/sensible/... We will read it and mark you down for inefficient/unclear/non-robust code.
- Reports: brief and "high level"; there's deliberately no space for anything more. Tell the story with punchy bullet points and provide references for details. A good wikipedia page, say, is ok; ChatGPT isn't.

Comments on coding, projects and reports

- Please read my comments on AI use! Talk/chat to anybody (human or non-human) but the work has to be your own and you have to understand it! Note that, in their infinite wisdom, the university enable Copilot by default, but don't blindly accept its suggestions.
- Gradescope is not a debugger! Passing our unit tests is not the same as your code being right/efficient/sensible/... We will read it and mark you down for inefficient/unclear/non-robust code.
- Reports: brief and "high level"; there's deliberately no space for anything more. Tell the story with punchy bullet points and provide references for details. A good wikipedia page, say, is ok; ChatGPT isn't.
- The maths isn't hard and no new mathematical concepts are covered. You may have forgotten the odd fact or genuinely not seen some things, in which case, go figure them out. It's what you'd do in real life.

Comments on coding, projects and reports

- Please read my comments on AI use! Talk/chat to anybody (human or non-human) but the work has to be your own and you have to understand it! Note that, in their infinite wisdom, the university enable Copilot by default, but don't blindly accept its suggestions.
- Gradescope is not a debugger! Passing our unit tests is not the same as your code being right/efficient/sensible/... We will read it and mark you down for inefficient/unclear/non-robust code.
- Reports: brief and "high level"; there's deliberately no space for anything more. Tell the story with punchy bullet points and provide references for details. A good wikipedia page, say, is ok; ChatGPT isn't.
- The maths isn't hard and no new mathematical concepts are covered. You may have forgotten the odd fact or genuinely not seen some things, in which case, go figure them out. It's what you'd do in real life.
- No screenshots, or long code listings in the report (unless code fragments help to clarify an object structure, say). Present results using clear graphics; discuss what they show.

Comments on coding, projects and reports

- Four "Analysis Exercises" worth 2.5% each. It may feel like you're spending "more time" on those. It's a carrot; the alternative is to offer 0%.

Comments on coding, projects and reports

- Four "Analysis Exercises" worth 2.5% each. It may feel like you're spending "more time" on those. It's a carrot; the alternative is to offer 0%.
- Would actually be interested in feedback on this. Observation from two years ago: people don't tend to spend a lot of time on the other (beautiful!) example sheet questions. (Following this rant, last year's lot were a lot better!).

Who are you?

Who knows (and is comfortable with):

- python
- matlab
- C++
- Chicken
  `https://esolangs.org/wiki/Chicken`
  see also `https://www.youtube.com/watch?v=yL_-1d9OSdk`

Who are you?

Who knows (and is comfortable with):

- python
- matlab
- C++
- Chicken
  `https://esolangs.org/wiki/Chicken`
  see also `https://www.youtube.com/watch?v=yL_-1d9OSdk`
- python's matplotlib
- plotting in matlab
- gnuplot

Who are you?

Who knows (and is comfortable with):

- python
- matlab
- C++
- Chicken
  `https://esolangs.org/wiki/Chicken`
  see also `https://www.youtube.com/watch?v=yL_-1d9OSdk`
- python's matplotlib
- plotting in matlab
- gnuplot
- windows command line (terminal)
- linux

How will you compute?

Are you planning to use

- the windoze machines in the computer cluster?
- your own laptop?

How will you compute?

Are you planning to use

- the windoze machines in the computer cluster?
- your own laptop?

If your own, which operating system are you planning to use?

- windoze
- linux
- mac

How will you compute?

Whatever you do, you must have

- An editor and a C++ compiler (gnu g++ is best). Feel free to use an IDE if you know how to use it.
- Word processing that allows you to create powerpoint-type presentations:
  - Powerpoint
  - Latex (e.g. using the beamer class; see the example provided)
    - Overleaf (online)
    - TeXStudio (on the PCs in the cluster)
    - Your own...

Today: Go to *"Getting started" on the Canvas page*

- **Compiling code:** Read the instructions in `compiling_in_cluster.pdf` on how to compile the `hello world.cpp` code in a windows terminal. (Or do this by other means, or on your own machine.)
- **Plotting:** Follow the (matlab-based) instructions in `plotting.pdf` or use your own favourite plotting package to prepare a simple line graph from the data listed there. You can also use the data in `fibonacci.dat`.
- **Writing powerpoint-style presentations:** Download the files
  - `analysis_exercise0.tex`
  - `fibonacci.pdf`
  - `fibonacci_validation.pdf`
  - `n_fn.dat`

  Put the last three into a sub-directory called `code`. Compile `analysis_exercise0.tex` using overleaf, TeXStudio or whatever you prefer. Alternatively, use your own favourite word processor to create a powerpoint-style presentation like the one shown in `analysis_exercise0.pdf`.

Today: Go to *"Getting started" on the Canvas page*

- **Upload files to Gradescope:** Upload the files
  - `analysis_exercise0.pdf`
  - `analysis_exercise0.cpp`

  to the course's Gradescope page for the (dummy) Analysis Project 0.

You can also download all the files as a zip file, `getting_started.zip`. Follow the instructions in `compiling_in_cluster.pdf` to create a directory (folder) in the `OneDrive` directory then unpack the zip file in that directory and work there.

Today: Go to *"Getting started" on the Canvas page*

- **Upload files to Gradescope:** Upload the files
  - `analysis_exercise0.pdf`
  - `analysis_exercise0.cpp`

  to the course's Gradescope page for the (dummy) Analysis Project 0.

You can also download all the files as a zip file, `getting_started.zip`. Follow the instructions in `compiling_in_cluster.pdf` to create a directory (folder) in the `OneDrive` directory then unpack the zip file in that directory and work there.

Nobody leaves this room until they can do all the above things!